

Numerical Algorithms

Problem set 4: Fast Fourier Transform

September 13, 2026

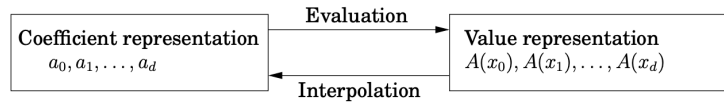
1 Polynomial multiplication and representation

- Let $A(x) = a_0 + a_1x + \cdots + a_dx^d$ and $B(x) = b_0 + b_1x + \cdots + b_dx^d$. Show that their product $C(x) = A(x) \cdot B(x) = c_0 + c_1x + \cdots + c_{2d}x^{2d}$ has coefficients

$$c_k = a_0b_k + a_1b_{k-1} + \cdots + a_kb_0 = \sum_{i=0}^k a_ib_{k-i}$$

(for $i > d$, take a_i and b_i to be zero). Argue that finding all the coefficients of the result will require $\Theta(d^2)$ time. Can you relate this to the junior school algorithm for integer multiplication?

- Show that a degree- d polynomial is uniquely characterized by its values at any $d + 1$ distinct points.
- Show that it gives us an alternative representation of polynomials. Fix any distinct points $x_0, \dots, x_d$. We can specify a degree- d polynomial $A(x) = a_0 + a_1x + \cdots + a_dx^d$ by either one of the following:
 - Its coefficients $a_0, a_1, \dots, a_d$
 - The values $A(x_0), A(x_1), \dots, A(x_d)$



- We can then write an algorithm for polynomial multiplication as:

Polynomial multiplication

Input: Coefficients of two polynomials, $A(x)$ and $B(x)$, of degree d

Output: Their product $C = A \cdot B$

Selection

Pick some points $x_0, x_1, \dots, x_{n-1}$, where $n \geq 2d + 1$

Evaluation

Compute $A(x_0), A(x_1), \dots, A(x_{n-1})$ and $B(x_0), B(x_1), \dots, B(x_{n-1})$

Multiplication

Compute $C(x_k) = A(x_k)B(x_k)$ for all $k = 0, \dots, n - 1$

Interpolation

Recover $C(x) = c_0 + c_1x + \dots + c_{2d}x^{2d}$

5. The selection and the multiplication steps are clearly linear time. What about evaluation and interpolation?

2 Divide and conquer

1. Here's an idea for how to pick the n points at which to evaluate a polynomial $A(x)$ of degree $\leq n - 1$. If we choose them to be positive-negative pairs, that is,

$$\pm x_0, \pm x_1, \dots, \pm x_{n/2-1},$$

then the computations required for each $A(x_i)$ and $A(-x_i)$ overlap a lot, because the even powers of x_i coincide with those of $-x_i$.

Show that,

$$A(x) = A_e(x^2) + xA_o(x^2),$$

where $A_e(\cdot)$, with the even-numbered coefficients, and $A_o(\cdot)$, with the odd-numbered coefficients, are polynomials of degree $\leq n/2 - 1$ (assume for convenience that n is even). Given *paired* points $\pm x_i$, the calculations needed for $A(x_i)$ can be recycled toward computing $A(-x_i)$:

$$\begin{aligned} A(x_i) &= A_e(x_i^2) + x_i A_o(x_i^2) \\ A(-x_i) &= A_e(x_i^2) - x_i A_o(x_i^2). \end{aligned}$$

In other words, evaluating $A(x)$ at n paired points $\pm x_0, \dots, \pm x_{n/2-1}$ reduces to evaluating $A_e(x)$ and $A_o(x)$ (which each have half the degree of $A(x)$) at just $n/2$ points, $x_0^2, \dots, x_{n/2-1}^2$.

2. Derive that, if we can apply the idea recursively, the time complexity of the algorithm evaluates to $O(n \log n)$.

3 FFT

1. Argue that the above divide and conquer strategy does not work for evaluation points in $\mathbb{R}$. The plus-minus trick only works at the top level of the recursion. To recurse at the next level, we need the $n/2$ evaluation points $x_0^2, x_1^2, \dots, x_{n/2-1}^2$ to be themselves plus-minus pairs. But a square cannot be negative.
2. Convince yourself, however, that the complex n^{th} roots of unity, the solutions to $z^n = 1$ will work. The n^{th} roots of unity, the complex numbers $1, \omega, \omega^2, \dots, \omega^{n-1}$, where $\omega = e^{2\pi i/n}$ are plus-minus paired. Show that if n is even,

- (a) The n th roots are plus-minus paired, $\omega^{n/2+j} = -\omega^j$.
- (b) Squaring them produces the $(n/2)$ nd roots of unity.

3. The resulting algorithm in the *Fast Fourier Transform* for polynomial evaluation

The fast Fourier transform (polynomial formulation)

function $FFT(A, \omega)$

Input: Coefficient representation of a polynomial $A(x)$
of degree $\leq n - 1$, where n is a power of 2
 ω , an n^{th} root of unity

Output: Value representation $A(\omega^0), \dots, A(\omega^{n-1})$

if $\omega = 1$: return $A(1)$

express $A(x)$ in the form $A_e(x^2) + xA_o(x^2)$

call $FFT(A_e, \omega^2)$ to evaluate A_e at even powers of ω

call $FFT(A_o, \omega^2)$ to evaluate A_o at even powers of ω

for $j = 0$ to $n - 1$:

 compute $A(\omega^j) = A_e(\omega^{2j}) + \omega^j A_o(\omega^{2j})$

return $A(\omega^0), \dots, A(\omega^{n-1})$

Convince yourself about its correctness.

4 Interpolation

To complete the polynomial multiplication algorithm we still need a fast method for polynomial interpolation at the roots of unity.

1. Verify that the two representations of a polynomial – the coefficient and the value representations – are related by a linear transformation:

$$\begin{bmatrix} A(x_0) \\ A(x_1) \\ \vdots \\ A(x_{n-1}) \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

2. The middle matrix M is called a *Vandermonde* matrix. Verify that if $x_0, \dots, x_{n-1}$ are distinct numbers, then M is invertible.
3. Verify that FFT multiplies an arbitrary n -dimensional vector—which we have been calling the *coefficient representation*—by the $n \times n$ matrix

$$M_n(\omega) = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^j & \omega^{2j} & \cdots & \omega^{(n-1)j} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{(n-1)} & \omega^{2(n-1)} & \cdots & \omega^{(n-1)(n-1)} \end{bmatrix} \begin{matrix} \leftarrow \text{row for } \omega^0 = 1 \\ \leftarrow \omega \\ \leftarrow \omega^2 \\ \vdots \\ \leftarrow \omega^j \\ \vdots \\ \leftarrow \omega^{n-1} \end{matrix}$$

4. Verify that $M_n(\omega)^{-1} = \frac{1}{n} M_n(\omega^{-1})$ (Hint: show that the columns are orthogonal), and that ω^{-1} is also an n^{th} root of unity. The inverse also can then be computed using the same FFT algorithm with powers of ω^{-1} as input.
5. Read about the efficient circuit representation of the FFT algorithm (the Butterfly algorithm).

The FFT algorithm is credited to John Cooley and John Tukey (1965), who devised it for computing some crucial steps in the US census. However, it was later discovered that British engineers used this algorithm for hand calculations in the 1930s, and Carl Friedrich Gauss had published a paper – written in Latin – on essentially the same idea in the early 1800s.

These problems have been adapted from a textbook on Algorithms by Dasgupta, Papadimitriou and Vazirani.